Sadqah Media Uploader: complete upload script

Full working source (not a snippet). Runs only in the owner's GitHub Actions workflow; uploads to the owner's own three channels. Source: github.com/imsv1301/islamic-content-automation/blob/main/yt/yt.py and .github/workflows/youtube.yml

yt/yt.py

```
"""YouTube tool for all three channels (official YouTube Data API v3 only).

State file (decrypted state/youtube.json.enc): {"client_id","client_secret","channels":{label:{"channel_id","refresh_token"}}}
Path from $YT_STATE (default /home/claude/.yt/youtube_state.json). Labels: sadqah, kids, riddle.

  yt.py upload   <label> <video.mp4> <meta.json> [thumb.jpg]
  yt.py comments <label>                 # all comment threads on the channel (JSON lines)
  yt.py reply    <label> <parentId> <text>
  yt.py stats    <label>                 # every upload with privacy + views/likes/comments
"""
import json, os, sys, urllib.error, urllib.parse, urllib.request

ST = json.load(open(os.environ.get("YT_STATE", "/home/claude/.yt/youtube_state.json")))
API = "https://www.googleapis.com/youtube/v3/"


def token(label):
    rt = ST["channels"][label]["refresh_token"]
    body = urllib.parse.urlencode(dict(client_id=ST["client_id"], client_secret=ST["client_secret"], refresh_token=rt,
grant_type="refresh_token")).encode()
    return json.load(urllib.request.urlopen("https://oauth2.googleapis.com/token", body))["access_token"]


def get(tok, path, **q):
    return json.load(urllib.request.urlopen(urllib.request.Request(API + path + "?" + urllib.parse.urlencode(q), headers=
{"Authorization": "Bearer " + tok})))


def upload(label, video, meta_f, thumb=None):
    tok = token(label); meta = json.load(open(meta_f)); size = os.path.getsize(video)
    req = urllib.request.Request("https://www.googleapis.com/upload/youtube/v3/videos?
uploadType=resumable&part=snippet,status", data=json.dumps(meta).encode(), method="POST",
        headers={"Authorization": "Bearer " + tok, "Content-Type": "application/json; charset=UTF-8", "X-Upload-Content-Type":
"video/mp4", "X-Upload-Content-Length": str(size)})
    loc = urllib.request.urlopen(req).headers["Location"]
    with open(video, "rb") as f:
        v = json.load(urllib.request.urlopen(urllib.request.Request(loc, data=f.read(), method="PUT", headers={"Authorization":
"Bearer " + tok, "Content-Type": "video/mp4"})))
    print("uploaded", v["id"], v["status"].get("privacyStatus"), "https://youtu.be/" + v["id"])
    if thumb:
        try:
            with open(thumb, "rb") as f:
                urllib.request.urlopen(urllib.request.Request(f"https://www.googleapis.com/upload/youtube/v3/thumbnails/set?
videoId={v['id']}", data=f.read(), method="POST", headers={"Authorization": "Bearer " + tok, "Content-Type": "image/jpeg"}))
            print("thumbnail set")
        except urllib.error.HTTPError as e:
            print("thumbnail failed", e.code, e.read()[:300])


def comments(label):
    tok = token(label); ch = ST["channels"][label]["channel_id"]
    try:
        r = get(tok, "commentThreads", part="snippet,replies", allThreadsRelatedToChannelId=ch, maxResults=100)
    except urllib.error.HTTPError as e:
        print("comments unavailable", e.code, "(made-for-kids channels have comments off)"); return
    for it in r.get("items", []):
        s = it["snippet"]["topLevelComment"]["snippet"]
        reps = [{"author": x["snippet"]["authorDisplayName"], "text": x["snippet"]["textOriginal"]} for x in it.get("replies",
{}).get("comments", [])]
        print(json.dumps({"thread": it["id"], "video": it["snippet"].get("videoId"), "author": s["authorDisplayName"], "text":
s["textOriginal"], "at": s["publishedAt"], "replies": reps}, ensure_ascii=False))


def reply(label, parent, text):
    tok = token(label)
    body = json.dumps({"snippet": {"parentId": parent, "textOriginal": text}}).encode()
    r = json.load(urllib.request.urlopen(urllib.request.Request(API + "comments?part=snippet", data=body, headers=
{"Authorization": "Bearer " + tok, "Content-Type": "application/json"})))
    print("replied", r["id"])


def stats(label):
    tok = token(label); ch = ST["channels"][label]["channel_id"]
    up = get(tok, "channels", part="contentDetails", id=ch)["items"][0]["contentDetails"]["relatedPlaylists"]["uploads"]
    ids = [i["contentDetails"]["videoId"] for i in get(tok, "playlistItems", part="contentDetails", playlistId=up,
maxResults=50).get("items", [])]
    if not ids: print("no uploads"); return
    for it in get(tok, "videos", part="snippet,status,statistics", id=",".join(ids)).get("items", []):
        s = it.get("statistics", {})
        print(it["id"], it["status"]["privacyStatus"], "views", s.get("viewCount"), "likes", s.get("likeCount"), "comments",
s.get("commentCount"), "|", it["snippet"]["title"])


if __name__ == "__main__":
```

```
    cmd, *a = sys.argv[1:]
    {"upload": upload, "comments": comments, "reply": reply, "stats": stats}[cmd](*a)
```

.github/workflows/youtube.yml

```
name: youtube
# Uploads / comments / replies / stats for the three channels. Tokens live only in state/youtube.json.enc
# (AES-256-CBC, pbkdf2) and are decrypted inside the job with secrets.MASTER_KEY.
# Videos to upload are read from the repo at `video_path` on branch `video_ref` (default: renders).
on:
  workflow_dispatch:
    inputs:
      action: { description: "upload | comments | reply | stats", required: true, default: stats }
      label: { description: "sadqah | kids | riddle", required: true, default: riddle }
      video_path: { description: "upload: path of the mp4 on video_ref", default: "" }
      video_ref: { description: "upload: branch holding the mp4", default: renders }
      meta_path: { description: "upload: metadata JSON path on main (snippet+status)", default: "" }
      parent_id: { description: "reply: comment thread id", default: "" }
      text: { description: "reply: reply text", default: "" }
jobs:
  run:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - if: inputs.action == 'upload'
        uses: actions/checkout@v4
        with: { ref: "${{ inputs.video_ref }}", path: renders, lfs: false }
      - name: Decrypt state
        env: { MASTER_KEY: "${{ secrets.MASTER_KEY }}" }
        run: |
          printf '%s' "$MASTER_KEY" > /tmp/k
          openssl enc -d -aes-256-cbc -pbkdf2 -iter 200000 -in state/youtube.json.enc -out /tmp/yt_state.json -pass file:/tmp/k
          rm /tmp/k
      - name: Run
        env:
          YT_STATE: /tmp/yt_state.json
          A: "${{ inputs.action }}"
          L: "${{ inputs.label }}"
          V: "${{ inputs.video_path }}"
          M: "${{ inputs.meta_path }}"
          P: "${{ inputs.parent_id }}"
          T: "${{ inputs.text }}"
        run: |
          case "$A" in
            upload) python3 yt/yt.py upload "$L" "renders/$V" "$M" | tee out.txt ;;
            reply) python3 yt/yt.py reply "$L" "$P" "$T" | tee out.txt ;;
            comments|stats) python3 yt/yt.py "$A" "$L" | tee out.txt ;;
            *) echo "unknown action $A"; exit 1 ;;
          esac
          rm -f /tmp/yt_state.json
      - uses: actions/upload-artifact@v4
        with: { name: "yt-${{ inputs.action }}-${{ inputs.label }}", path: out.txt }
```